CPYTHON INTERNALS

Фрагменти й копії

Як Python марнує пам'ять

Технічний огляд механізмів копіювання, прихованих операцій та фрагментації пам’яті в інтерпретаторі CPython.

Сумець Світлана

2026

Вступ: чому пам’ять «зникає»

> Python часто створює більше об’єктів і копій, ніж ми усвідомлюємо. Код працює, помилок немає - але пам’ять фрагментується, а продуктивність повільно падає.

Аналогія з містом

Без планування місто розростається хаотично: з’являються прогалини й мертві зони. Так само поводиться пам’ять без контролю алокацій.

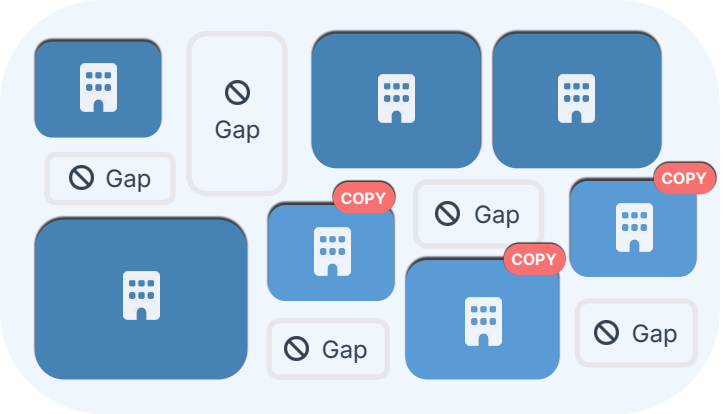
Непомітне накопичення

Зайві копії та фрагменти не ламають код, але забирають RAM і ускладнюють роботу GC. Проблема росте тихо - поки не стає критичною.

Мета та фокус

Розберемо роботу CPython (pymalloc, GC). Навчимося бачити, коли створюються копії, без необхідності переписувати архітектуру.

ФРАГМЕНТАЦІЯ: МІСЬКА ЗАБУДОВА



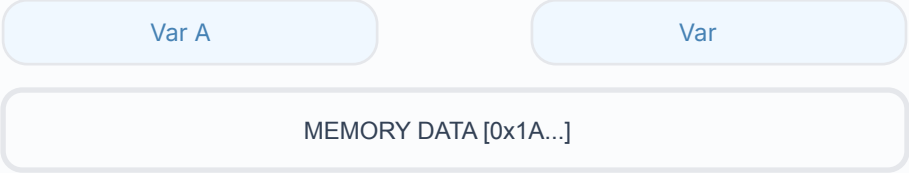
- Зайнята пам'ять (Корисна)
- Фрагментована пам’ять (Мертва зона)
- Зайві непродумані копії об'єктів

Копіювання: Zero-copy vs Python

Zero-copy (C, Rust)

ІДЕАЛЬНИЙ СВІТ

- Дані передаються без фактичного дублювання в пам'яті
- Робота напряму з вказівниками та буферами
- Мінімальне навантаження на процесор



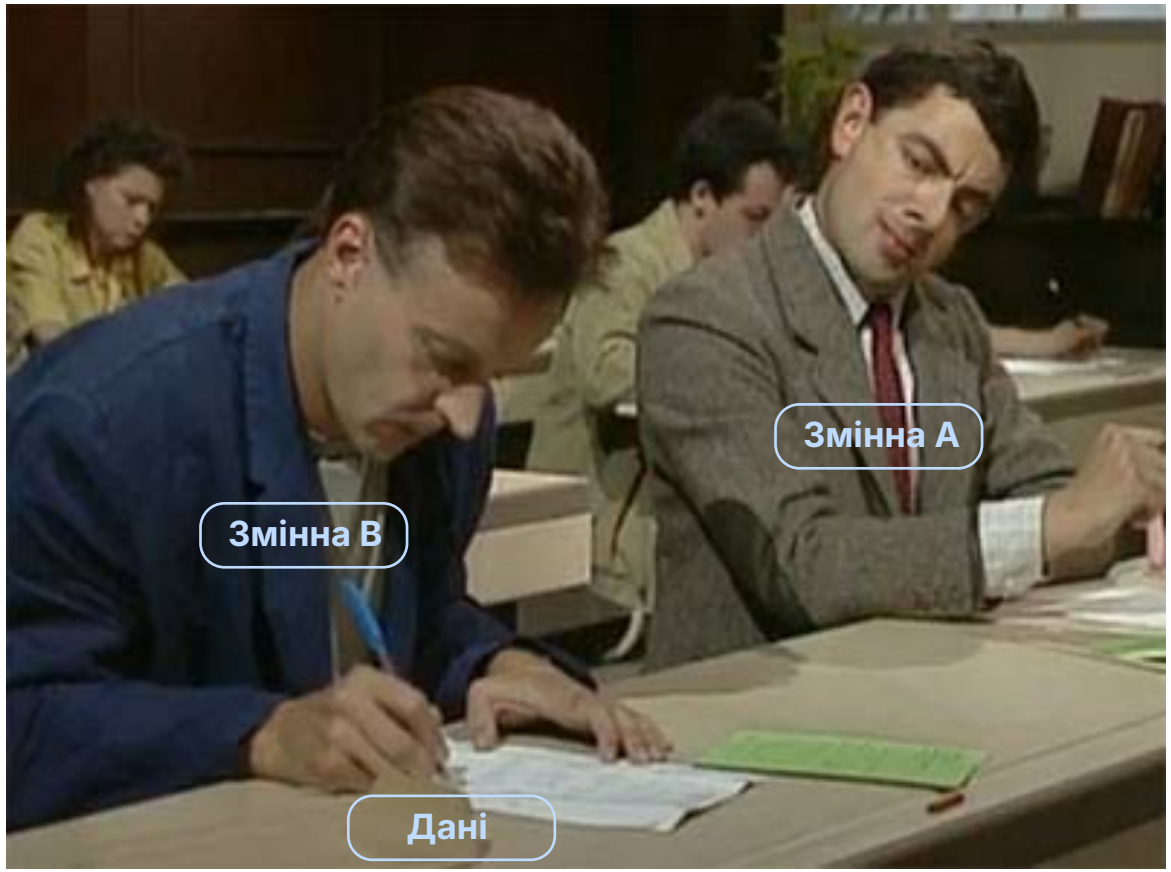
Python (CPython)

РЕАЛЬНІСТЬ

- Майже кожна операція створює новий об'єкт
- Всюди копіювання або створення посилань
- Ціна: алокації, фрагментація, робота GC

Копіювання

- Копіюються не сутності, а зв’язки між ними
- Копіювання - це ілюзія незалежності
- Два імені, одна сутність
- Копія - це часто лише інший шлях до того самого
- Копіювання - це компроміс між зручністю та ілюзією окремості
- Копіювання - обіцянка спільної відповідальності
- Копіювання - це розширення кола впливу, а не меж об’єкта

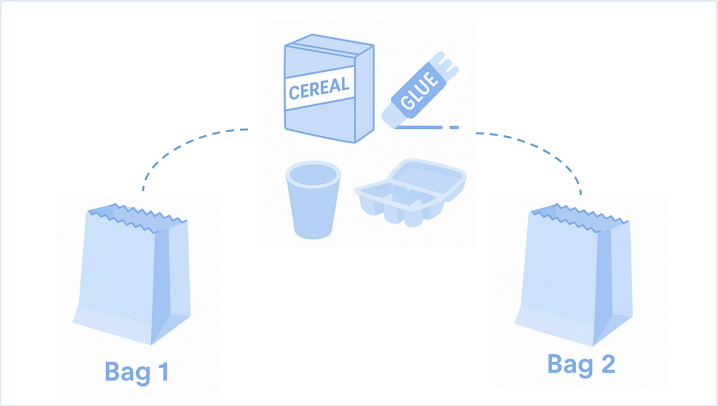


😂 Дві змінні - один “ой”.
Виглядає незалежно. Ламається синхронно..

Поверхневе копіювання: copy.copy()

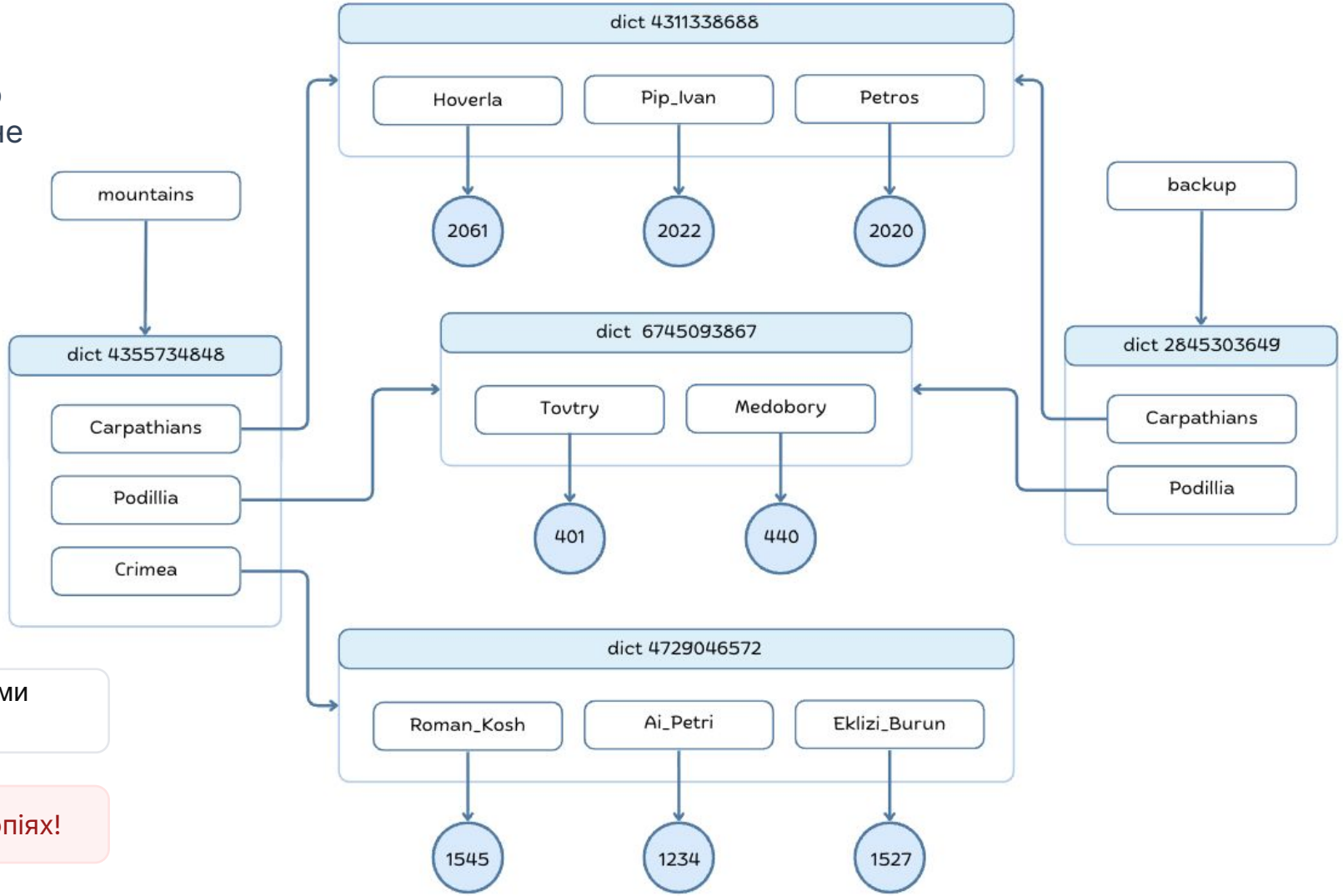
Як це працює?

Функція copy() створює новий контейнер лише верхнього рівня. Вкладені об'єкти не дублюються, а просто отримують нові посилання.



backup і mountains - різні об’єкти, але зі спільними вкладеними словниками.

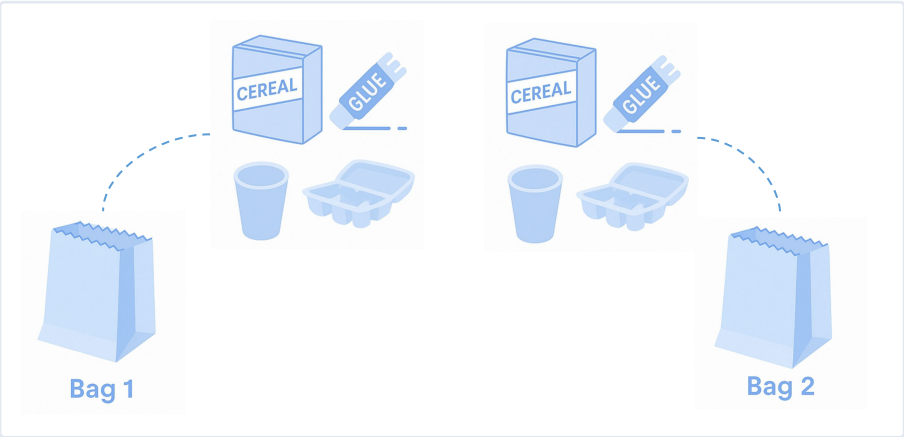
Зміни у вкладених словниках видно в обох копіях!



Глибоке копіювання: copy.deepcopy()

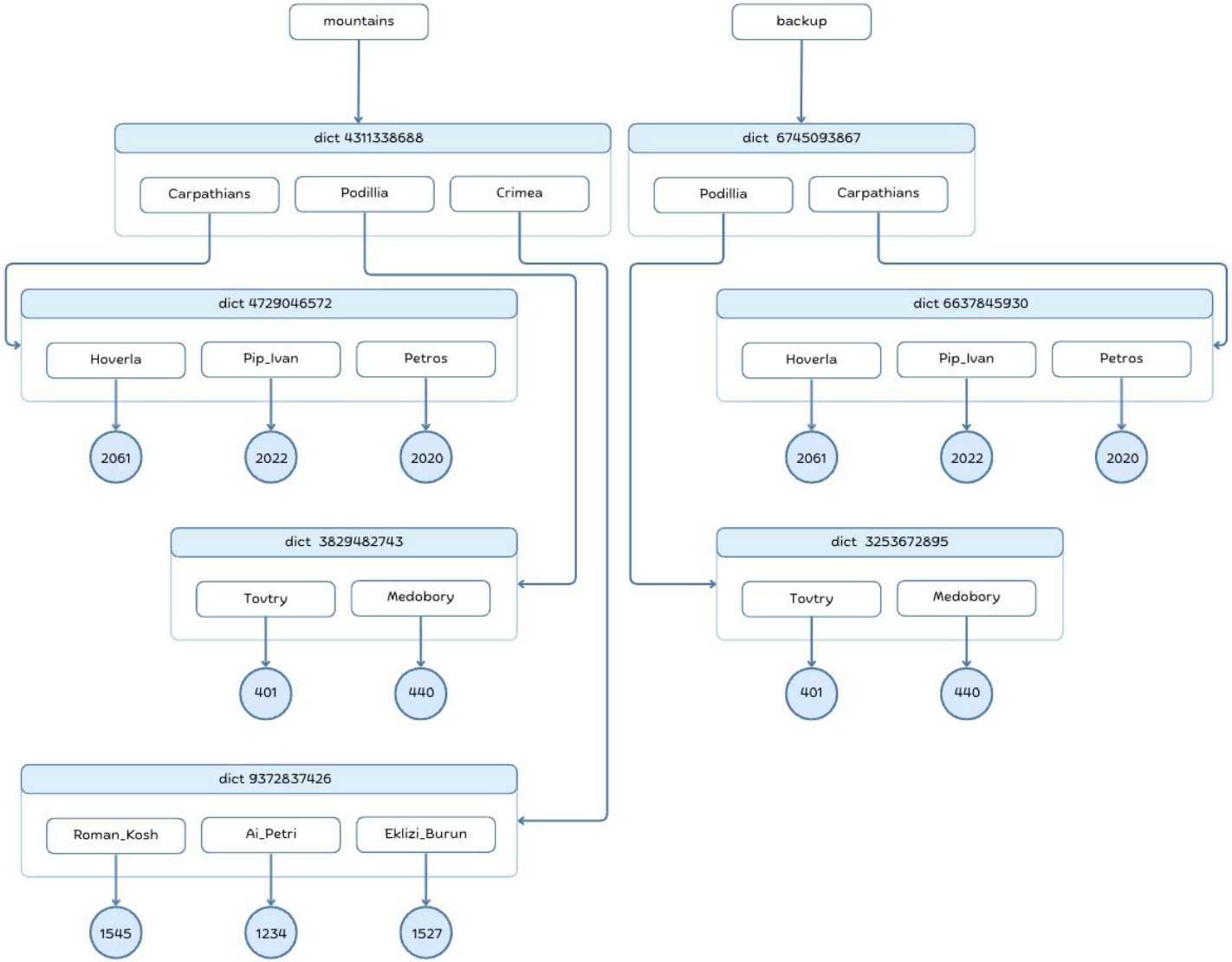
Як це працює?

Функція deepcopy() проходить по всьому дереву об'єктів, створюючи нові копії для кожного рівня вкладеності. Оригінал і копія стають повністю незалежними.



Перевага: Повна ізоляція даних: зміни в одній структурі ніколи не зламають іншу.

Треба бути обережним із захаращенням пам'яті



Рекурсивні структури та Золотий перетин

Стратегія копіювання

copy(): Ефективно для пласких структур. Лише верхній рівень.

deepcopy(): Необхідно для глибокої ізоляції та циклічних посилань.

ЦИКЛІЧНЕ ПОСИЛАННЯ

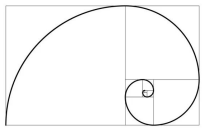
```
initial = [1]
initial.append(initial)  # Список додає сам себе всередину
```

```
Result: [1, [...]]
```

```
print(initial[1] is initial)
# True -> Нескінченна рекурсія!
```

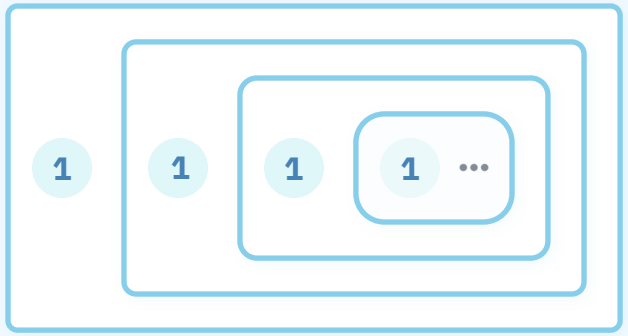
$$\varphi = 1 + \frac{1}{\varphi} \quad \text{and} \quad \varphi = 1 + \cfrac{1}{1 + \cfrac{1}{1 + \cfrac{1}{1 + \cdots}}}$$

Математична аналогія: Золотий перетин



Структура [1, [...]] нагадує ланцюговий дріб золотого перетину: самоподібність, що йде у нескінченність.

MEMORY VISUALIZATION



Увага: Поверхнева копія (copy()) не впорається з цим і створить некоректну структуру. Тут потрібен лише deepcopy()!

Приховане копіювання

> Багато звичних операцій у Python створюють нові об'єкти «під капотом», навіть якщо це виглядає безпечно по пам’яті.

Зрізи (Slicing)

Операції list[:], str[1:5] завжди створюють новий об'єкт-контейнер. Для змінюваних типів це поверхнева копія.

Приведення типів

Конструктори list(), dict(), set() створюють нову структуру пам'яті, копіюючи посилання з джерела.

Метод .copy()

Це просто зручний шорткат (alias) для функції copy.copy(). Працює ідентично - поверхневе копіювання.

Розпакування (Unpacking)

Синтаксис [*data] або {**data} розгортає елементи в новий контейнер, створюючи новий об'єкт списку чи словника.

АНАТОМІЯ СЛАЙСІНГУ

```
a = [10,20,30,40]
b = a[1:3]
# b is a new list object!
```

ЗМІННА	АДРЕСА (ID)	ЗНАЧЕННЯ
a	0x7f...010	10 20 30 40
	↓	
b	0x7f...088	20 30

Новий об'єкт (алокація)

Порада: Замість list(x) часто краще працювати з самим x або використовувати ітератори, якщо повна копія не потрібна.

Сортування та Конкатенація

Сортування

`sorted()` Завжди створює новий список.
Витрачає пам'ять на повну копію даних перед початком сортування.

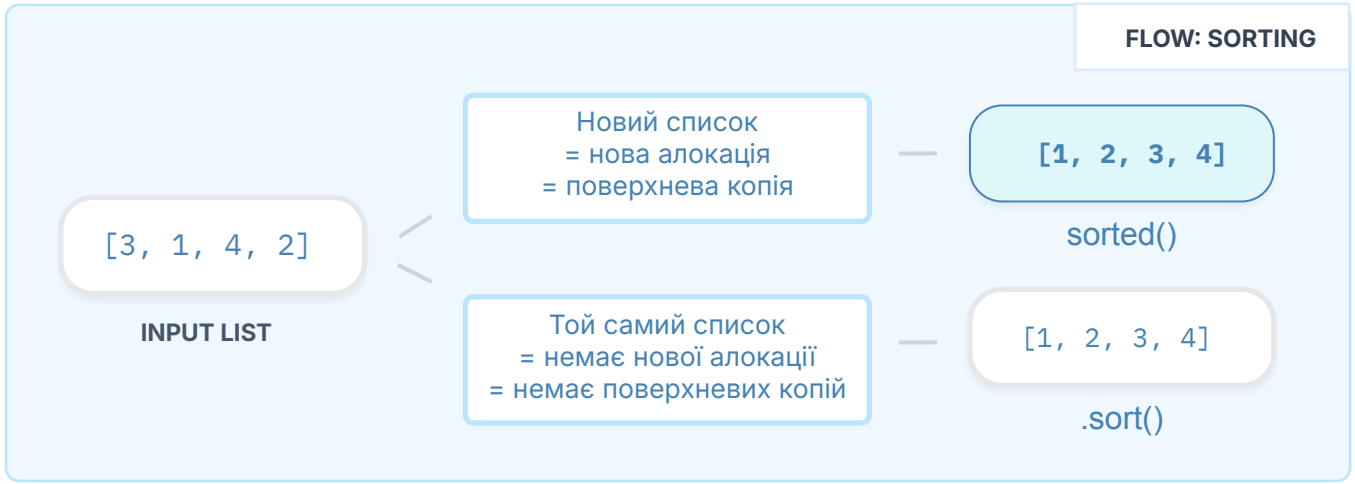
`.sort()` Сортує «на місці» (In-place).
Але алгоритм Timsort все одно використовує тимчасові буфери пам'яті.

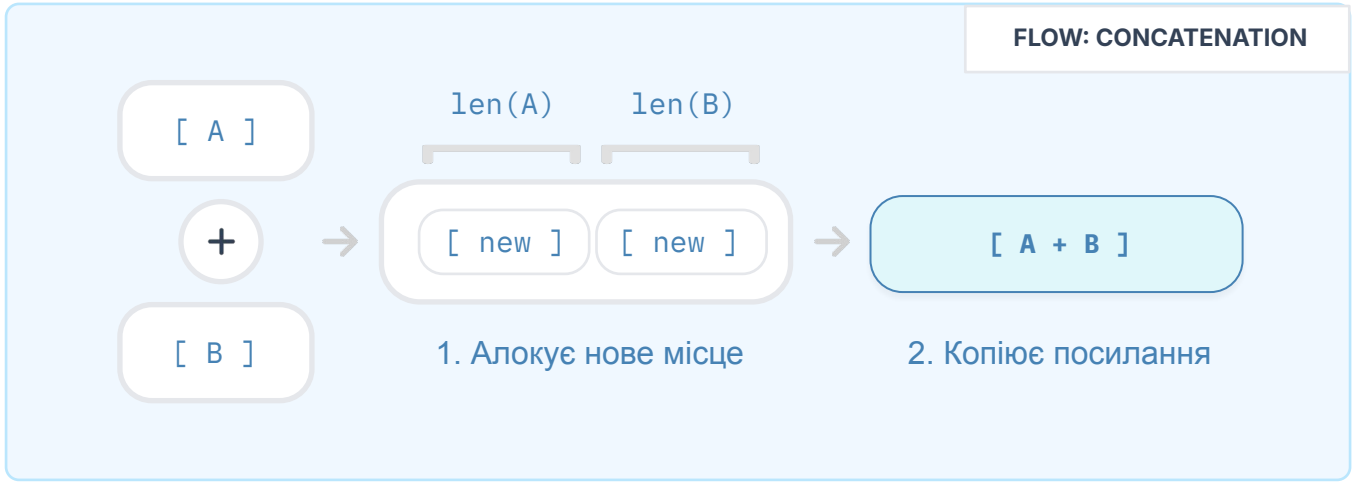
Тимчасова алокація O(n) у гіршому випадку для .sort(). In-place ≠ zero-allocation

Конкатенація (+)

Операція a + b не просто "дописує" дані. Python створює новий контейнер розміром len(a) + len(b), а потім копіює туди всі посилання.

Уникайте конкатенації у великих циклах - це квадратична складність!





Фрагментація та pymalloc

Проблема «дірявої» Пам'яті

Фрагментація - проблема, коли вільна пам'ять «розсипана» дрібними шматками. CPython не може повернути сторінку пам'яті ОС, доки вона не стане повністю порожньою.

РІСТ RSS УПОВІЛЬНЕННЯ

Ієрархія pymalloc

Arena 256 KB. Великий шмат від ОС

Pool 4 KB. Частина арени для блоків

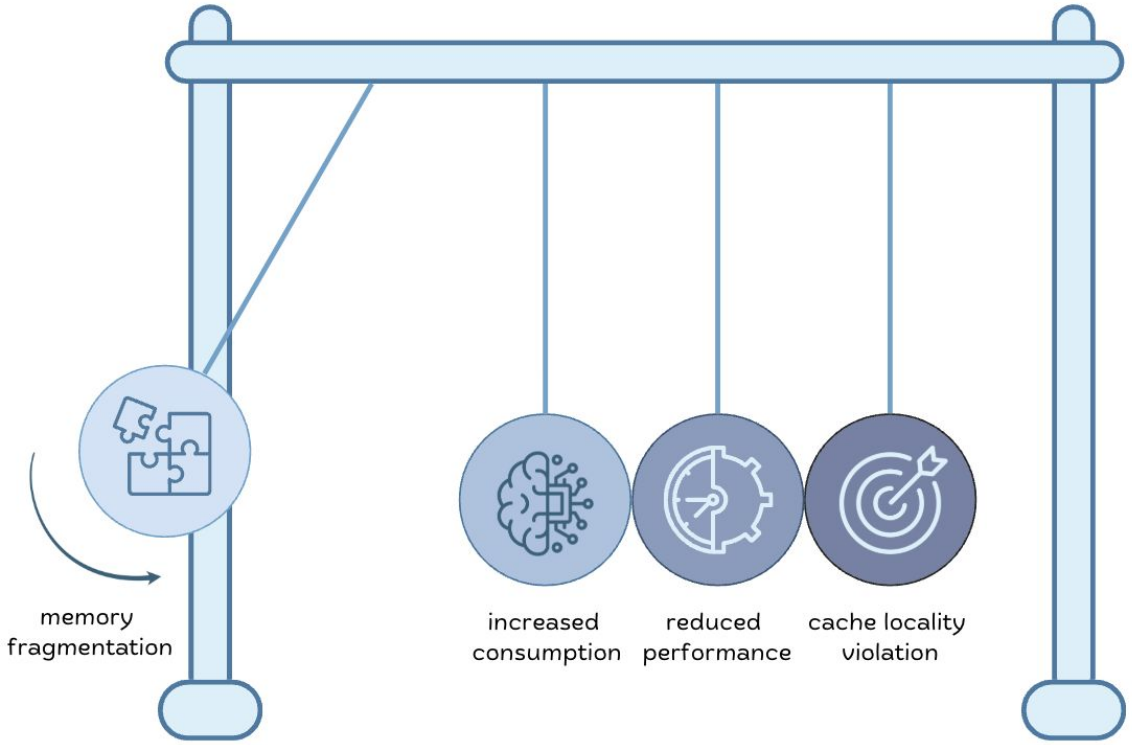
Block. Фіксований розмір (напр. 32B) в пулі для даних

>_ ДІАГНОСТИКА

Змінні середовища для статистики:

```
PYTHONMALLOC=debug PYTHONMALLOCSTATS=1 python
1_memory/test.py 2>&1 | tee malloc_stats.txt
```

ВІЗУАЛІЗАЦІЯ ПРОБЛЕМИ



Фрагментація ≠ нестача пам’яті. Навіть коли RAM достатньо, хаотичні алокації погіршують кеш-локальність, збільшують RSS і знижують продуктивність.

Генератори: «ліниві» обчислення

Стратегія «на вимогу»

Генератор не зберігає дані. Він зберігає інструкцію, як отримати наступний елемент.

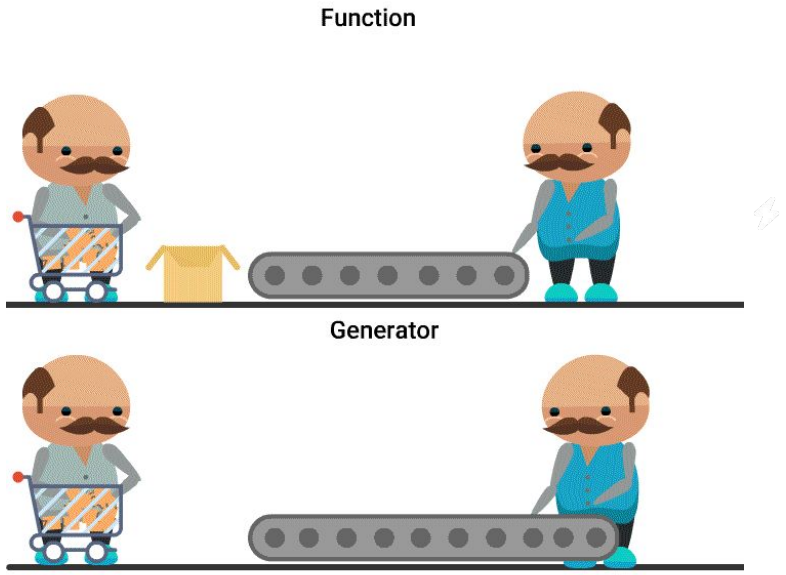
ЖИТТЄВИЙ ЦИКЛ ГЕНЕРАТОРА

Створення
Зберігається лише стан та посилання на код

Запит (next)
Виконання коду до першого yield

Видача значення
Повертає 1 елемент і "заморожує" стан

Очікування
Пам'ять вільна від даних



Великі файли

Читання CSV/Logs рядок за рядком без завантаження в RAM

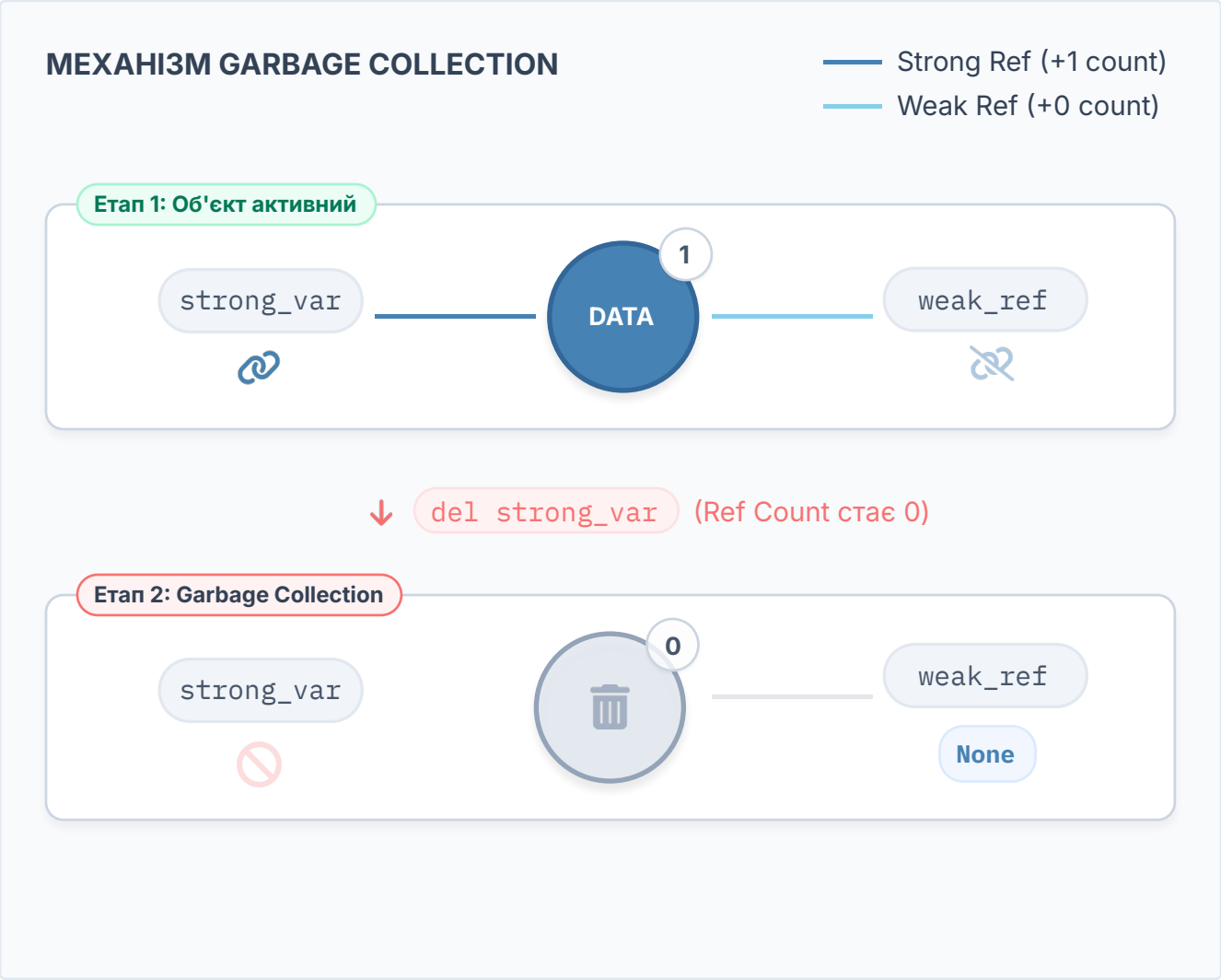
Стрімінг

Обробка безкінечних потоків даних в реальному часі

Пайплайни

Передача даних між функціями без проміжних списків

Слабкі посилання (Weak References)



Сценарії використання

Кешування

WeakValueDictionary. Об'єкти видаляються з кешу автоматично, коли зникають у програмі, запобігаючи витокам пам'яті.

Дерева та Графи

Використовується для посилань «дитина → батько». Розриває циклічні посилання, дозволяючи GC коректно звільняти пам'ять.

Observer Pattern

Підписка на події. Якщо «слухач» видаляється, система перестає надсилати йому сповіщення без явного відписування.

Обмеження: Не працює з вбудованими типами int, str, tuple напряму.

Слоти (__slots__)

Стандартний об'єкт

__dict__

ДИНАМІЧНА ХЕШ-ТАБЛИЦЯ

Hash(Key) →	Value
'name'	'Kharkiv'
'pop'	1,300,000
Empty (Overhead)	
Empty (Overhead)	

obj.name

Scan...

Витрати пам'яті

Оптимізований Об'єкт

__slots__

ФІКСОВАНИЙ МАСИВ (C-STRUCT)

Object Header + Descriptor

Offset 0	Offset 1
'Kharkiv'	1,300,000

obj.name

Direct pointer

Економія ресурсів

Проблема списку словників

Ієрархія об'єктів у пам'яті

СПОЖИВАННЯ ПАМ'ЯТІ (1M ЗАПИСІВ)

267.45 MB

Величезні накладні витрати на 1 мільйон об'єктів словників та 3 мільйони об'єктів ключів.

Фрагментація Купи

Дрібні об'єкти розкидані по пам'яті, що спричиняє «дірки».

Низька Кеш-Локальність

Процесор марнує цикли на очікування даних з різних ділянок RAM замість послідовного читання.

Порівняння структур даних

СТАНДАРТНИЙ ПІДХІД

List[dict]

267.45 MB

↑ Base Line

HASH MAP STRUCTURE

"id"	→	1001
"name"	→	"test"
"age"	→	25

Дублювання ключів

ОПТИМІЗАЦІЯ №1

List[namedtuple]

153.01 MB

↓ -43% Пам'яті

TUPLE (COMPACT)

0	→	1001
1	→	"test"
2	→	25

Фіксовані індекси

МАКСИМАЛЬНА ЕФЕКТИВНІСТЬ

List[dataclass] slots=True

145.38 MB

↓ -46% Пам'яті

C-STRUCT ARRAY

offset:0	→	1001
offset:1	→	"test"
offset:2	→	25

Без __dict__

Тест на мільйоні записів з полями: id (int), name (str), age (int).

Джерело: pympler.asizeof

Висновки: як писати ефективний код

Свідоме копіювання

СТРАТЕГІЯ

Чітко розрізняйте, коли потрібен copy(), а коли deepcopy().

Якщо об'єкт незмінний або вкладеність не важлива - уникайте глибокого копіювання.

Використовуйте copy() для плоских структур

Ітератори та Генератори

ПАМ'ЯТЬ

Замість створення повних списків через list(), використовуйте генератори.

Обробляти дані «ліниво», стрімінгово, не завантажуючи весь масив у RAM, - завжди ефективніше

yield замість append() у великих циклах

Оптимізація операцій

ШВИДКОДІЯ

Уникайте прямої конкатенації у циклах, оскільки вона створює проміжні об'єкти.

Для рядків краще "".join(), для списків - extend().

Менше тимчасових об'єктів = менше GC

Діагностика

КОНТРОЛЬ

Фрагментація накопичується непомітно. Перевіряйте алокації та стан пам'яті.

Використовуйте інструменти профілювання та змінні середовища.

```
PYTHONMALLOC=debug
```

Часто мікро-оптимізації та вчасне профілювання запобігає перелаштуванню архітектури та виникненню «прихованих проблем» як-от фрагментація.

Що почитати?

- Copy-Deepcopy
- Copy-on-write (Instagram Engineering)
- PEP 445
- Estimating memory usage
- Data doesn't fit in memory
- Python Memory Management: Best Practices for Performance